

Linker Description File

// stm32_ls.ld

```
ENTRY(Reset_Handler)
```

```
MEMORY
```

```
{
    FLASH (rx) : ORIGIN = 0x08000000, LENGTH = 1024K
    SRAM (rwx) : ORIGIN = 0x20000000, LENGTH = 128K
}
```

```
SECTIONS
```

```
{
    /* Code + Read-only data */
    .text :
    {
        *(.isr_vector)      /* Interrupt vector table */
        *(.text*)           /* Program code */
        *(.rodata*)         /* Read-only data */
        . = ALIGN(4);        /* Align to next word boundary */
        _etext = .;          /* End of text section */
    } > FLASH

    /* Initialized data (copied from FLASH to SRAM at startup) */
    .data :
    {
        _sdata = .;         /* Start of data section */
        *(.data*)
        . = ALIGN(4);        /* Align to next word boundary */
        _edata = .;         /* End of data section */
    } > SRAM AT > FLASH

    /* Uninitialized data (zeroed at startup) */
    .bss :
    {
        _sbss = .;          /* Start of BSS */
        *(.bss*)
        *(COMMON)
        _ebss = .;          /* End of BSS */
    } > SRAM
}
```

Linker Script Memory Section:-

```
Lenovo@Lenevo MINGW64 /d/Argyan Technology/STM32_Embedded_Course/ARM_Programming_Course/Task_Project/my_workspace (main)
$ ls
Makefile led.c led.h main.c main.h main_log stm32_ls.ld stm32_startup.c

Lenovo@Lenevo MINGW64 /d/Argyan Technology/STM32_Embedded_Course/ARM_Programming_Course/Task_Project/my_workspace (main)
$ make
arm-none-eabi-gcc -c -mcpu=cortex-m4 -mthumb -std=gnu11 -Wall -O0 -o main.o main.c
arm-none-eabi-gcc -c -mcpu=cortex-m4 -mthumb -std=gnu11 -Wall -O0 -o led.o led.c
arm-none-eabi-gcc -c -mcpu=cortex-m4 -mthumb -std=gnu11 -Wall -O0 -o stm32_startup.o stm32_startup.c

Lenovo@Lenevo MINGW64 /d/Argyan Technology/STM32_Embedded_Course/ARM_Programming_Course/Task_Project/my_workspace (main)
$ ls
Makefile led.c led.h led.o main.c main.h main.o main_log stm32_ls.ld stm32_startup.c stm32_startup.o

Lenovo@Lenevo MINGW64 /d/Argyan Technology/STM32_Embedded_Course/ARM_Programming_Course/Task_Project/my_workspace (main)
$ arm-none-eabi-gcc -nostdlib -T stm32_ls.ld *.o -o final.elf

Lenovo@Lenevo MINGW64 /d/Argyan Technology/STM32_Embedded_Course/ARM_Programming_Course/Task_Project/my_workspace (main)
$ ls
Makefile final.elf led.c led.h led.o main.c main.h main.o main_log stm32_ls.ld stm32_startup.c stm32_startup.o

Lenovo@Lenevo MINGW64 /d/Argyan Technology/STM32_Embedded_Course/ARM_Programming_Course/Task_Project/my_workspace (main)
$ arm-none-eabi-objdump.exe -h final.elf

final.elf:      file format elf32-littlearm

Sections:
Idx Name          Size      VMA       LMA       File off  Algn
 0 .text          000006a8  08000000  08000000  00010000  2**2
   CONTENTS, ALLOC, LOAD, READONLY, CODE
 1 .data          00000001  20000000  080006a8  00020000  2**0
   CONTENTS, ALLOC, LOAD, DATA
 2 .bss           00000054  20000004  080006a9  00020004  2**2
   ALLOC
 3 .comment       00000049  00000000  00000000  00020001  2**0
   CONTENTS, READONLY
 4 .ARM.attributes 0000002e  00000000  00000000  0002004a  2**0
   CONTENTS, READONLY
```

// reset handler implementation

```
void Reset_Handler(void){
    /* Copy .data section to SRAM*/
    uint32_t size = &_amp;edata - &_amp;sdata;
    uint8_t *pDst = (uint8_t*)&sdata; // SRAM
    uint8_t *pSrc = (uint8_t*)&etext; // FLASH

    for(uint32_t i = 0; i < size; i++){
        *pDst++ = *pSrc++;
    }

    /* Init the .bss section to zero in SRAM */
    size = &_amp;ebss - &_amp;sbss;
    for(uint32_t i = 0; i < size; i++){
        *pDst++ = 0;
    }

    /* call main() */
    main();
}
```